

Digital Adaptive Twin-Driven Planning For Multi-robot Navigation in GPS Denied Environment

Capstone Project Report

Submitted in partial fulfilment for the award of the degree of

Bachelor of Technology

In

**ELECTRONICS AND COMMUNICATION ENGINEERING
(AI & CYBERNETICS)**

Submitted to

VIT BHOPAL UNIVERSITY (M. P.)

Submitted by

BADRIKANATH PRAHARAJ -22BAC10032

SREERAM MV - 22BAC10004

Under the Supervision of

Dr. Om Prakash Pahari

SCHOOL OF ELECTRICAL & ELECTRONICS ENGINEERING

VIT BHOPAL UNIVERSITY

BHOPAL (M.P.)-466114

March –2026

© VIT Bhopal University - Year



VIT BHOPAL UNIVERSITY, BHOPAL

SCHOOL OF ELECTRICAL & ELECTRONICS ENGINEERING

DECLARATION

I hereby declare that the Dissertation entitled "Digital Adaptive Twin-Driven Planning For Multi-robot Navigation in GPS-Denied Environment" is my own work conducted under the supervision of Dr. Om Prakash pahari, Associate Professor, school of Electrical and Electronics at VIT Bhopal University, Bhopal.

I further declare that to the best of my knowledge, this report does not contain any part of work that has been submitted for the award of any degree either in this university or in other university / Deemed University without proper citation.

Badrikanath Praharaj
22BAC10032

Sreeram MV
22BAC10004



VIT BHOPAL UNIVERSITY, BHOPAL
SCHOOL OF ELECTRICAL & ELECTRONICS ENGINEERING

CERTIFICATE

This is to certify that the work embodied in this Capstone project report entitled **"Digital Adaptive Twin-Driven Planning For Multi-robot Navigation in GPS Denied Environment"** has been satisfactorily completed by **Mr. Badrikanath Praharaj Registration No 22BAC10032 & Mr. Sreeram MV Registration No 22BAC10004** in the School of Electrical & Electronics Engineering of at VIT Bhopal University, Bhopal. This work is a bonafide piece of work, carried out under my guidance in the School of Electrical & Electronics Engineering for the partial fulfilment of the degree of Bachelor of Technology.

Dr. Om Prakash Pahari

Supervisor

Forwarded by

Dr. SOUMITRA KESHARI NAYAK

Program Chair

Approved by

Dr. SURESH M

Dean, SEEE

Acknowledgment

In the first place, I would like to express my gratitude to **Dr. Om Prakash Pahari** for his supervision, advice, and guidance from the very early stage of this thesis work as well as for giving me extraordinary experiences throughout the work. Above all and the most needed, he provided me with unflinching encouragement and support in various ways. His words have always inspired me to work efficiently and comprehensively. I would like to thank him for his constant encouragement that enabled me to grow as a person. His presence has definitely improved me as a human being.

I express my gratitude to "**Dr. M Suresh**," Senior Associate Professor & Dean, School of Electrical and Electronics Engineering, VIT Bhopal University, for providing me with all the help and allowing me to work in the laboratory with no time limits.

I shall ever remain indebted to "**Prof. (Dr.) Satish Kumar Modh**", Vice Chancellor, VIT Bhopal University, for providing me with institutional and administrative facilities during my project work at VIT.

I would like to express my sincere gratitude to all the faculty members, staff, colleagues, and well-wishers who, directly or indirectly, extended their support and cooperation throughout the course of this work. Their encouragement, assistance, and goodwill have been of immense value in the successful completion of this project.

Badrikanath Praharaj
22BAC10032

Sreeram MV
22BAC10004

Abstract

Autonomous navigation for a team of robots in a mining environment is difficult because the robots must do more than simply move toward their own goals; they also have to avoid obstacles, prevent collisions, work within time limits, and succeed as a coordinated team in a cluttered arena. In this project, that challenge is addressed through a three-robot multi-agent reinforcement learning framework built in PyBullet, where the environment includes a $50\text{ m} \times 50\text{ m}$ mining arena, static obstacles such as pillars, ore piles, and equipment, differential-drive robot dynamics, collision handling, goal generation, frozen-on-goal behavior, and observations formed from goal-relative features, LiDAR readings, robot motion, and teammate information.

What gives this work its strength is that it is not treated as a simple training problem, but as a carefully designed learning system in which the environment, reward logic, and training strategy support one another. The training pipeline uses a MADDPG-based centralized-training, decentralized-execution setup with twin critics, delayed actor updates, soft target updates, adaptive exploration noise, dead-actor recovery, and phase-aware replay sampling to improve learning stability. At the same time, the reward design strongly favors actual task completion by combining progress-based shaping with goal bonuses, team bonuses, collision penalties, and timeout penalties, while the eight-phase smooth curriculum advances only when both individual and team success meet the required mastery level. Taken together, the project develops a structured and practical pipeline for multi-robot coordination in a mining-inspired setting, showing that reliable team behavior depends not only on the learning algorithm, but also on how well the simulator, curriculum, replay strategy, and reward design are aligned.

In addition to the learning framework, this work incorporates a hardware-oriented perception and state estimation pipeline to support deployment in GPS-denied environments. A multi-modal sensor setup consisting of LiDAR, IMU, and wheel encoders is integrated within a ROS 2 architecture, where an Extended Kalman Filter (EKF) is used to fuse sensor data and provide a reliable pose estimate. This fused state is utilized by a SLAM module for mapping and localization, while a Gazebo-based mining environment is used to validate system performance under realistic conditions. Furthermore, an Adaptive Digital Twin framework is developed to establish a connection between simulation and real-world behavior, enabling continuous updates, improved prediction, and enhanced navigation reliability.

List of Figures

Figure No.	Caption / Title	Page No.
Figure 1.1:	System Architecture	2
Figure 3.2:	Occupancy Grid Map	20
Figure 3.3:	Gazebo Simulation Environment	22
Figure 3.4:	Blender Mining Terrain Model	23
Figure 3.5:	Drift Accumulation (Sim vs Real)	25
Figure 3.6:	Localization & Sensing Errors Comparison	26
Figure 4.1:	MADDPG Architecture	30
Figure 4.2:	Mining Arena	31
Figure 4.3:	Reward & Curriculum Visualization	32
Figure 4.4:	Distributed Learning Architecture	33
Figure 5.1:	Average Success Rate	36
Figure 5.2:	Success Composition	37
Figure 5.3:	Robot Fairness	38
Figure 5.4:	End Reason Distribution	39
Figure 5.5:	Collision Severity	40

List of Tables

Table No.	Caption / Title	Page No.
Table 3.1:	Sensor Hardware Specifications	15
Table 3.2:	EKF Configuration Parameters	17
Table 3.3:	SLAM Toolbox Parameters	18
Table 3.4:	Adaptive Digital Twin Architecture	24

List of Symbols & Abbreviations

A	Ampere
ARM	Association Rule Mining
DIC	Dynamic Item-Set Counting
DM	Data Mining
DHP	Direct Hashing and Pruning

Note: All Abbreviations and symbols that appear in the report should be listed alphabetically. First give all roman symbols, then Greek symbols. Order: ASCII ordering, to the extent possible. Subscripts and superscripts should be listed separately if these are not an intrinsic part of the variable name.

Table of Contents

Contents	Page No.
Abstract	vi
List of Figures	vii
List of Tables	viii
List of Symbols & Abbreviations	ix
1 INTRODUCTION	1
1.1 Software	2
1.1.1 Motivation	
1.1.2 General Theory Related to the Topic	
1.1.3 Objective	
1.1.4 Brief Methodology	
2 LITERATURE REVIEW	5
2.1 Software	
3 HARDWARE INTEGRATION, SENSOR FUSION, AND ROS 2 ARCHITECTURE	7
3.1 Overview	
3.2 Sensor Hardware Suite	
3.3 ROS 2 Communication Architecture and Data Flow	
3.4 Extended Kalman Filter for Sensor Fusion	
3.4.1 Motivation	
3.4.2 State Vector and Nonlinear Motion Model	
3.4.3 Prediction and Update Steps	
3.4.4 Implementation: robot_localization Package	
3.5 SLAM: Simultaneous Localisation and Mapping	
3.5.1 Algorithm Selection — slam_toolbox	
3.5.2 Pose-Graph Architecture	
3.5.3 Configuration	
3.5.4 Generated Map — Results and Analysis	
3.5.5 Map Quality and Potential Improvements	
3.6 Gazebo Simulation Environment	
3.6.1 Platform and Rationale	
3.6.2 Mining Tunnel Environment Design	
3.6.4 Sim-to-Real Transfer Considerations	
3.8 Adaptive Digital Twin Architecture	
3.8.1 Definition	
3.8.2 Five-Layer Architecture	
3.8.3 Bidirectional Feedback Loop	

- 3.9 Simulation versus Real-World Performance Analysis
- 3.10 Chapter Summary and Statement

4 PROBLEM FORMULATION AND PROPOSED METHODOLOGY

9

- 4.1 Problem Formulation
- 4.2 Proposed Methodology
 - 4.2.1 Multi-Agent Learning Module
 - 4.2.2 Simulation and Environment Module
 - 4.2.3 Reward and Curriculum Module
 - 4.2.4 Training Infrastructure Module
 - 4.2.5 Diagnostics and Evaluation Module

5 RESULTS AND DISCUSSION

5.1 Software

6 CONCLUSION AND FUTURE SCOPE

15

- 6.1 Conclusion
 - 6.1.1 Software
- 6.2 Future Scope
 - 6.2.1 Software

1. INTRODUCTION

1.1 Software

1.1.1 Motivation

Multi-robot navigation in a mining-like environment is difficult because the robots must do more than travel to their own targets; they must move safely in a cluttered arena, avoid collisions, and finish the task as a coordinated team within limited episode duration. In this project, the task becomes more demanding because three robots operate together inside a 50 m $\times$ 50 m simulated mining space containing pillars, ore piles, equipment, walls, and realistic goal-based completion rules. The work was motivated by the need to study not only whether robots can learn to move, but whether they can learn stable and cooperative behavior under realistic sensing, obstacle interaction, and team-level success conditions.

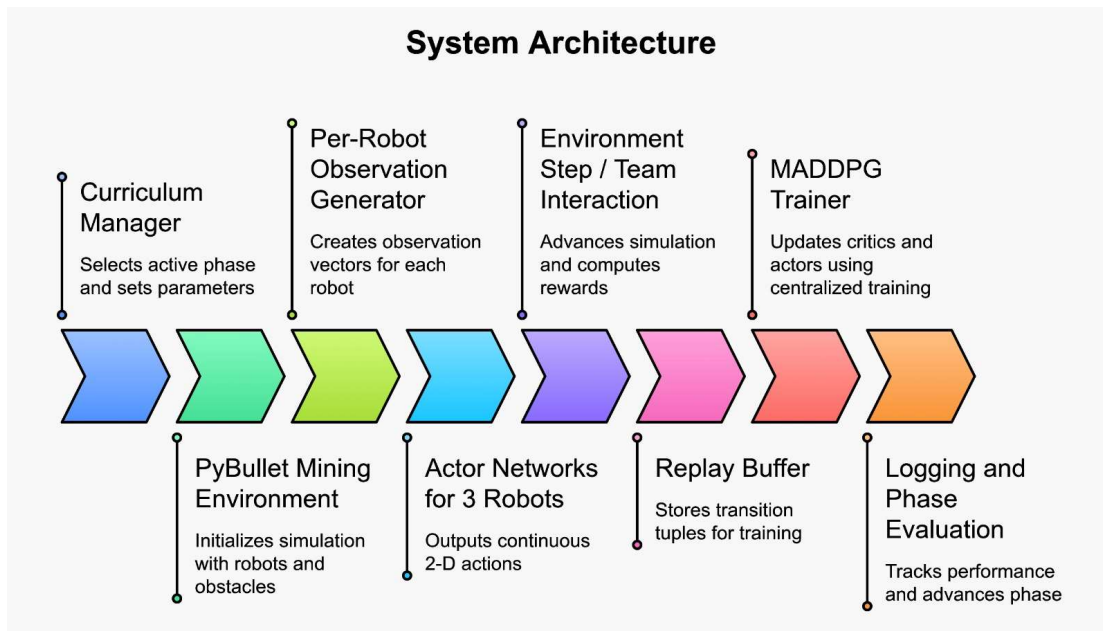


Fig 1: System Architecture

1.1.2 General Theory Related to the Topic

The project is based on centralized training and decentralized execution, where agents are trained with shared team-level information but act independently using their own observations at execution time. This idea is implemented through a MADDPG-based framework in which each robot uses an actor network to produce actions, while critic networks evaluate the joint observations and actions of all agents during training. The trainer also includes twin critics, delayed actor updates, target smoothing, soft target updates, and gradient clipping to improve learning stability.

From the environment side, each robot receives a compact observation made up of goal-relative features, 16 LiDAR readings, velocity terms, and teammate-relative information. Actions are continuous and correspond to normalized linear and angular velocity commands, which are scaled to the robot's motion limits before being applied in simulation. Since the environment

includes collision handling, frozen-on-goal behavior, timeout conditions, and progressively harder phases, the problem becomes a cooperative decision-making problem rather than a simple path-planning task.

1.1.3 Objective

The main objective of this work is to develop a multi-agent reinforcement learning framework that enables three robots to navigate safely and cooperatively in a mining-inspired environment and reach assigned goals under realistic constraints. A second objective is to build the surrounding training system carefully, so that learning remains stable as task difficulty increases across phases. The project also aims to align reward design, curriculum progression, replay sampling, and diagnostics so that the learned behavior reflects true team success rather than weak solutions such as repeated partial completion or timeout-heavy episodes.

1.1.4 Brief Methodology

The methodology followed in this work is summarized below:

- A PyBullet mining environment was used with three differential-drive robots, fixed obstacles, collision logic, goal spawning, and frozen-on-goal semantics.
- Each robot learned from goal, LiDAR, velocity, and teammate information and produced continuous motion commands for linear and angular movement.
- Training was carried out using a MADDPG-based setup with shared architecture, twin critics, delayed policy updates, soft target updates, and adaptive exploration noise.
- A goal-first reward structure was used, combining progress-based shaping with goal bonuses, team bonuses, collision penalties, and timeout penalties to keep learning focused on full task completion.
- An eight-phase smooth curriculum was applied, and advancement was allowed only after rolling mastery in both individual and team success.
- To improve efficiency and robustness, the system used phase-aware replay sampling, success-weighted storage, catastrophe handling, parallel rollouts, and detailed logging for later analysis.
- A hardware-oriented perception and state estimation pipeline was developed to support deployment in GPS-denied environments.
- A multi-modal sensor suite consisting of LiDAR, IMU, and wheel encoders was integrated using ROS 2, enabling real-time data acquisition and communication between system components.
- An Extended Kalman Filter (EKF) was implemented to fuse sensor data and provide a reliable and continuous pose estimate by reducing noise and drift from individual sensors.
- The fused state estimate was utilized by a SLAM module to perform mapping and localization within the mining environment.
- A Gazebo-based simulation environment was used to validate the hardware pipeline under realistic conditions, ensuring consistency between simulated and real-world behavior.

- An Adaptive Digital Twin framework was developed to establish a connection between simulation and real-world deployment, enabling continuous updates, improved prediction, and enhanced navigation reliability.

2. Literature Survey

2.1 Software

One of the most relevant papers for the software foundation of this project is "**Multi-Agent Actor-Critic for Mixed Cooperative-Competitive Environments**" (2017), which introduces the MADDPG framework. The paper explains why standard reinforcement learning becomes difficult in multi-agent settings, mainly because each agent sees a non-stationary environment created by the learning behavior of other agents, and it proposes a centralized critic with decentralized policies to address that issue. This is directly useful for our project because the implemented trainer is also built around a CTDE-style MADDPG variant for three robots, where each robot acts from its own observation while training uses joint information from all agents. In simple terms, this paper gives the software-level learning backbone for coordinated multi-robot decision making in our system.

A second key paper is "**Addressing Function Approximation Error in Actor-Critic Methods**" (2018), which presents the TD3 method. The core contribution of this paper is showing that actor-critic methods can suffer from overestimation bias and unstable value learning, and it addresses this using twin critics, delayed policy updates, and noisy target actions. This matters strongly for our work because the current training code already uses twin critics, TD3-style target minimization, policy smoothing, and delayed actor updates to improve stability in continuous-control multi-robot learning. So while MADDPG provides the multi-agent structure, this paper contributes the software ideas that make the learning loop more reliable and less prone to critic-driven instability.

A third important reference is "**Curriculum Learning for Reinforcement Learning Domains: A Framework and Survey**" (2020). This paper argues that difficult reinforcement learning problems can often be learned more effectively when tasks are arranged in a meaningful progression instead of being attacked at full difficulty from the start. That idea is very closely reflected in our software design, where training progresses through multiple phases from Easy1 to VHard, and phase advancement occurs only after rolling mastery conditions are satisfied for both individual and team success. The value of this paper for our project is that it supports the logic behind our staged training pipeline, showing that curriculum design is not just a heuristic add-on but a principled way to improve learning efficiency and robustness.

A fourth useful paper is "**Learning to Fly — a Gym Environment with PyBullet Physics for Reinforcement Learning of Multi-agent Quadcopter Control**" (2021). The paper highlights a common gap in robotics research: highly realistic simulators are often hard to scale for learning, while lightweight RL environments are often too simplistic, and it proposes a Gym-like multi-agent environment built on PyBullet to balance realism and training usability. This connects well with our project because our environment is also a custom multi-robot simulation

built in PyBullet, with realistic collisions, obstacle geometry, LiDAR-style perception, and continuous action control designed specifically for reinforcement learning. The main contribution of this paper to our work is that it validates the software idea of using a physics-based simulator not merely for visualization, but as a structured learning environment where algorithm design and robotic realism can meet.

A fifth paper that fits the software subsection very well is "**ROS-PyBullet Interface: A Framework for Reliable Contact Simulation and Robot Data Collection in Python**" (2023). This work emphasizes full-physics simulation with PyBullet, integration with the ROS ecosystem, and easier transfer from simulation to real robotic setups. That is highly relevant to our project because the broader software vision also involves building a system that can move beyond isolated training code toward a more deployable robotics pipeline, where simulation, sensing, coordination, and middleware support must all work together. In that sense, this paper contributes the bridge between simulation software and real robotic system integration, which is especially useful for projects targeting mines, tunnels, or other GPS-denied environments where testing in simulation is essential before deployment.

A sixth important aspect of this project is the integration of hardware-aware perception and state estimation, which complements the software learning framework. For this, concepts from sensor fusion and probabilistic robotics were incorporated, particularly the use of the Extended Kalman Filter (EKF) as described in standard robotics literature such as "**A Generalized Extended Kalman Filter Implementation for the Robot Operating System**" (2014). This approach enables the fusion of LiDAR, IMU, and wheel encoder data to produce a reliable and continuous pose estimate in GPS-denied environments. In addition, SLAM techniques inspired by works such as "Real-Time Loop Closure in 2D LiDAR SLAM" (2016) were utilized to generate occupancy grid maps for navigation. These methods are integrated within a ROS 2 and Gazebo-based simulation pipeline, forming a bridge between perception and decision-making. Furthermore, the concept of a Digital Twin, based on foundational ideas in "Digital Twin: Manufacturing Excellence through Virtual Factory Replication" (2014), is extended in this project into an Adaptive Digital Twin framework, where simulation and real-world data are continuously aligned. Together, these contributions strengthen the system by ensuring that learning-based policies operate on accurate, real-time state information and remain applicable for real-world robotic deployment.

3. Hardware Integration, Sensor Fusion, and ROS 2 Architecture

3.1 Overview

This chapter documents the hardware-level individual contributions to the Digital Adaptive Twin-Driven Planning for Multi-Robot Navigation in GPS-Denied Environments project. The specific contributions covered herein include: (i) the design and integration of the multi-modal sensor suite; (ii) the Extended Kalman Filter (EKF)-based sensor fusion pipeline; (iii) the Simultaneous Localisation and Mapping (SLAM) implementation; (iv) the Gazebo Classic simulation environment representing the mining tunnel; and (v) the Adaptive Digital Twin (ADT) architecture that bridges virtual simulation and physical deployment.

The hardware infrastructure is built entirely within the ROS 2 Humble Hawksbill middleware framework, which provides topic-based inter-process communication, a lifecycle-managed node architecture, and hardware abstraction layers enabling near-seamless sim-to-real transfer.

3.2 Sensor Hardware Suite

Table 3.1: Sensor hardware specifications and ROS 2 topic assignments.

The robot platform integrates a multi-modal sensor suite comprising a 2D LiDAR, an Inertial Measurement Unit (IMU), differential-drive wheel encoders, and an optional monocular camera. Each modality contributes a complementary type of environmental information, and their outputs are fused through the EKF pipeline (Section 3.4). Table 3.1 summarises the

Sensor	Model / Type	ROS 2 Topic	Rate (Hz)	Key Specification
2D LiDAR	RPLiDAR A1/A2	/scan	10	360° FOV, 12 m range, 1° resolution
IMU	6-DOF MEMS IMU	/imu/data	100	±16 g accel; ±2000 °/s gyro
Wheel Encoders	Quadrature encoder	/odom	50	Differential-drive, 500 CPR
Camera (opt.)	USB / CSI Camera	/camera/image_raw	30	RGB 1080p, 90° FOV
Compute	RPi 4B / Jetson Nano	—	—	4 GB RAM, Ubuntu 22.04

hardware specifications and associated ROS 2 topic assignments.

3.3 ROS 2 Communication Architecture and Data Flow

All sensor drivers, state-estimation nodes, SLAM, and navigation stack components communicate via the ROS 2 publish-subscribe paradigm over a DDS (Data Distribution Service) transport layer. Quality-of-Service profiles are tuned per data stream: /scan and /imu/data use BEST_EFFORT reliability with depth-1 history to minimise latency; /odom uses RELIABLE with depth-10 to prevent data loss during EKF prediction; /map uses TRANSIENT_LOCAL durability so that newly initialised nodes receive the latest map immediately.

3.4 Extended Kalman Filter for Sensor Fusion

3.4.1 Motivation

Reliable state estimation is the foundational requirement for both SLAM accuracy and safe navigation in GPS-denied environments. No single sensor provides a complete and noise-free estimate: LiDAR scans are sparse in time (10 Hz), wheel odometry drifts under slip, and IMU angular velocity integrates noise that accumulates as unbounded heading error. Sensor fusion through an Extended Kalman Filter (EKF) resolves this by combining multiple noisy measurements into a statistically optimal state estimate that is more accurate and more robust than any individual sensor stream.

3.4.2 State Vector and Nonlinear Motion Model

The EKF maintains a six-dimensional state vector representing the full 2D pose and velocity of the differential-drive robot in the fixed world frame:

$$x = [x, y, \theta, \dot{x}, \dot{y}, \dot{\theta}]^T \in \mathbb{R}^6$$

where x, y are the robot's planar coordinates, θ is the heading angle, and $\dot{x}, \dot{y}, \dot{\theta}$ are their time derivatives. The discrete-time state-transition function propagates the state using the unicycle kinematic model. Process noise $w_k \sim N(0, Q_k)$ models unmodelled dynamics including wheel slip and vibration.

3.4.3 Prediction and Update Steps

Prediction (at each IMU timestamp, 100 Hz):

$$\begin{aligned}\hat{x}_{\{k|k-1\}} &= f(\hat{x}_{\{k-1|k-1\}}, u_k) \\ P_{\{k|k-1\}} &= F_k P_{\{k-1|k-1\}} F_k^T + Q_k\end{aligned}$$

Update (at each available sensor measurement):

$$\begin{aligned}
K_k &= P_{\{k|k-1\}} H_k^T (H_k P_{\{k|k-1\}} H_k^T + R_k)^{-1} \\
\hat{x}_{\{k|k\}} &= \hat{x}_{\{k|k-1\}} + K_k (z_k - h(\hat{x}_{\{k|k-1\}})) \\
P_{\{k|k\}} &= (I - K_k H_k) P_{\{k|k-1\}}
\end{aligned}$$

3.4.4 Implementation: robot_localization Package

The EKF is implemented using the robot_localization ROS 2 package (ekf_node). The node accepts asynchronous measurements from multiple sensor topics and integrates each measurement as it arrives, without requiring temporal synchronisation between sensor streams.

Parameter	Value	Description
frequency	50 Hz	EKF filter and publish rate
odom0	/odom	Wheel odometry input
odom0_config	[T, T, F, F, F, T, ...]	Fuse x, y, yaw from odometry
imu0	/imu/data	IMU input
imu0_config	[F,F,F, T,T,T, F,F,T,...]	Fuse angular vel., yaw from IMU
process_noise_covariance	Diag[0.05, 0.05, 0.01, ...]	Motion model uncertainty Q
initial_estimate_covariance	Diag[0.1, 0.1, 0.05, ...]	Initial state uncertainty P ₀

Table 3.2: Key EKF configuration parameters (robot_localization ekf_node).

3.5 SLAM: Simultaneous Localisation and Mapping

3.5.1 Algorithm Selection — slam_toolbox

For the GPS-denied underground mining environment, 2D LiDAR-based SLAM was selected as the primary mapping modality due to its accuracy in structured corridor geometries, computational tractability on embedded compute platforms, and direct compatibility with the ROS 2 Nav2 stack. The slam_toolbox package implements pose-graph SLAM with nonlinear least-squares optimisation via the Ceres Solver, provides online SLAM for real-time mapping, loop-closure detection via scan-to-map correlation, and a map serialisation capability for session persistence.

3.5.2 Pose-Graph Architecture

The SLAM algorithm maintains a pose graph $G = (V, E)$, where each vertex $v_i \in V$ represents a robot pose at time i , and each edge $e_{ij} \in E$ encodes a spatial constraint derived from either consecutive odometry increments (odometry edges) or scan-matching alignment (loop-closure edges). The occupancy grid map is derived by raycasting all accumulated LiDAR scans into the world frame using the optimised pose-graph vertex positions.

3.5.3 Configuration

Parameter	Value	Effect
minimum_travel_distance	0.5 m	Minimum odometry travel before new node added
minimum_travel_heading	0.5 rad	Minimum heading change before new node added
map_update_interval	5.0 s	Grid regeneration interval
loop_search_maximum_distance	3.0 m	Radius for closure candidate search
correlation_search_space_dimension	0.5 m	Scan matcher search space
resolution	0.05 m/cell	Occupancy grid resolution

Table 3.3: slam_toolbox configuration parameters for the mining environment.

3.5.4 Generated Map — Results and Analysis

The SLAM pipeline was executed within the Gazebo simulation environment to validate map quality prior to hardware deployment. The resulting occupancy grid map is a 570×880 cell grid at 0.05 m resolution (28.5 m $\times$ 44 m coverage). Pixel values encode: white = confirmed free space, dark grey = obstacles/walls, mid-grey = unexplored/unknown regions.

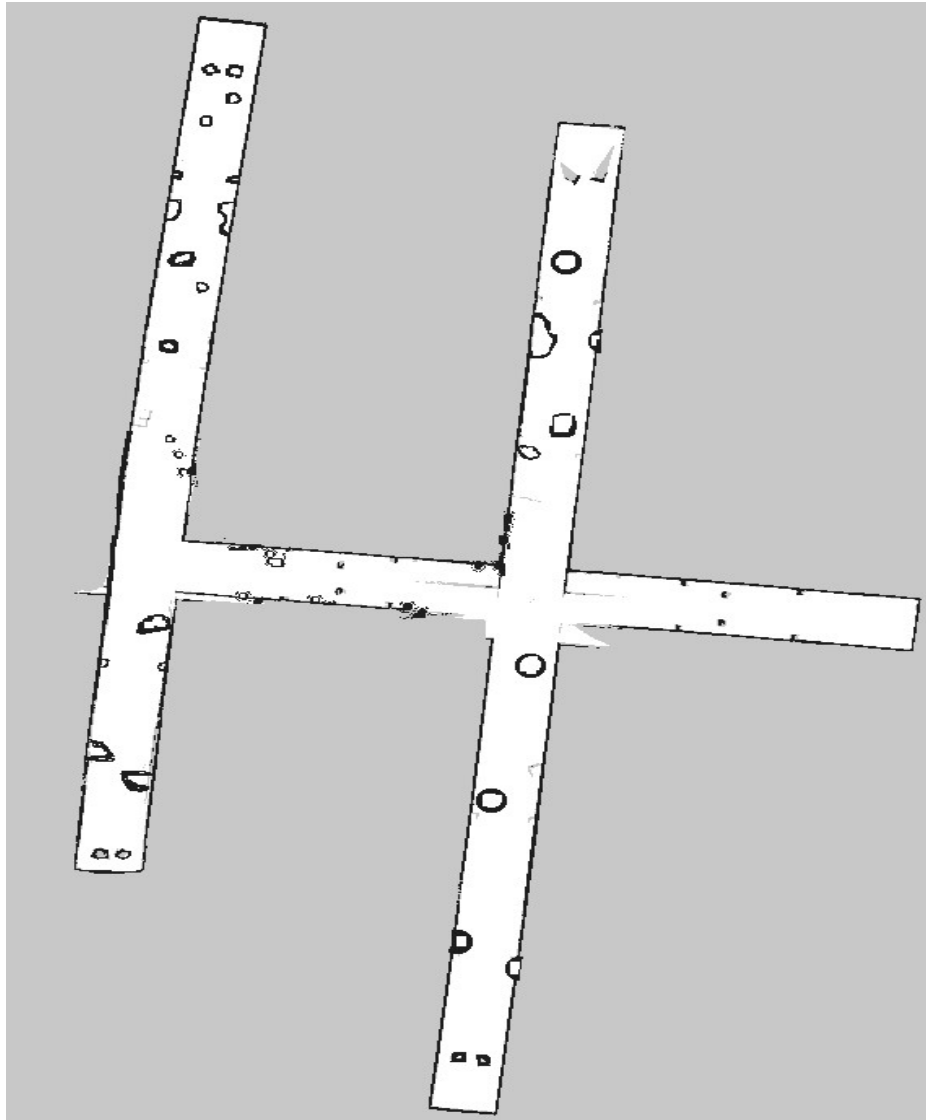


Figure 3.2: Occupancy grid map generated by *slam_toolbox* from Gazebo simulation traversal. White: free space. Dark: obstacles and walls. Mid-grey: unexplored regions. Map resolution: 0.05 m/cell; grid size: 570×880 cells.

The map exhibits clearly delineated free corridors separated by sharp, continuous obstacle boundaries, consistent with successful scan-matching in the corridor-dominated mining geometry.

3.5.5 Map Quality and Potential Improvements

- Adaptive resolution: Reducing cell resolution from 0.05 m to 0.025 m in narrow sections could sharpen wall-edge representation.
- IMU pre-integration: Incorporating IMU measurements between odometry updates for the SLAM pose prior would reduce corner-artifact frequency.
- Multi-session mapping: `slam_toolbox`'s lifelong mapping mode would allow incremental map extension across multiple exploration runs.
- 3D SLAM: For environments with significant vertical structure, replacing 2D LiDAR SLAM with a 3D variant (e.g., LIO-SAM) would capture ceiling-height variation.

3.6 Gazebo Simulation Environment

3.6.1 Platform and Rationale

Gazebo Classic (version 11) was selected as the physics simulation platform for algorithm validation. It provides rigid-body dynamics via the ODE engine, a ray-cast sensor plugin architecture fully compatible with ROS 2 through the `gazebo_ros_pkgs` bridge, and an SDF world-definition system for procedural environment construction.

3.6.2 Mining Tunnel Environment Design

The simulated world models a segment of an underground mine tunnel with narrow orthogonal corridors, low ambient lighting, and distributed cylindrical obstacles representing ore boulders and mining equipment. Corridor width is approximately 3 m, consistent with a single-lane mine drift. Lighting uses low-lux point sources at fixed ceiling positions, replicating the visually challenging conditions of an active mine.

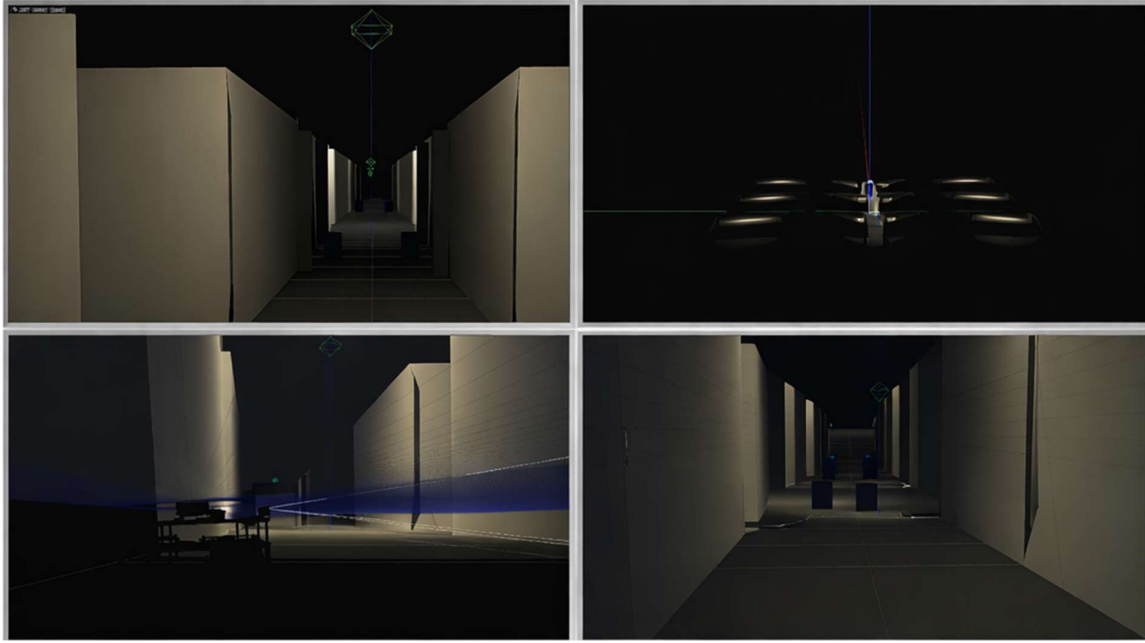


Figure 3.3: Gazebo simulation environment showing multiple perspectives of the mining tunnel, including first-person navigation view, LiDAR-based perception, overhead layout, and robot localization within the simulated environment.

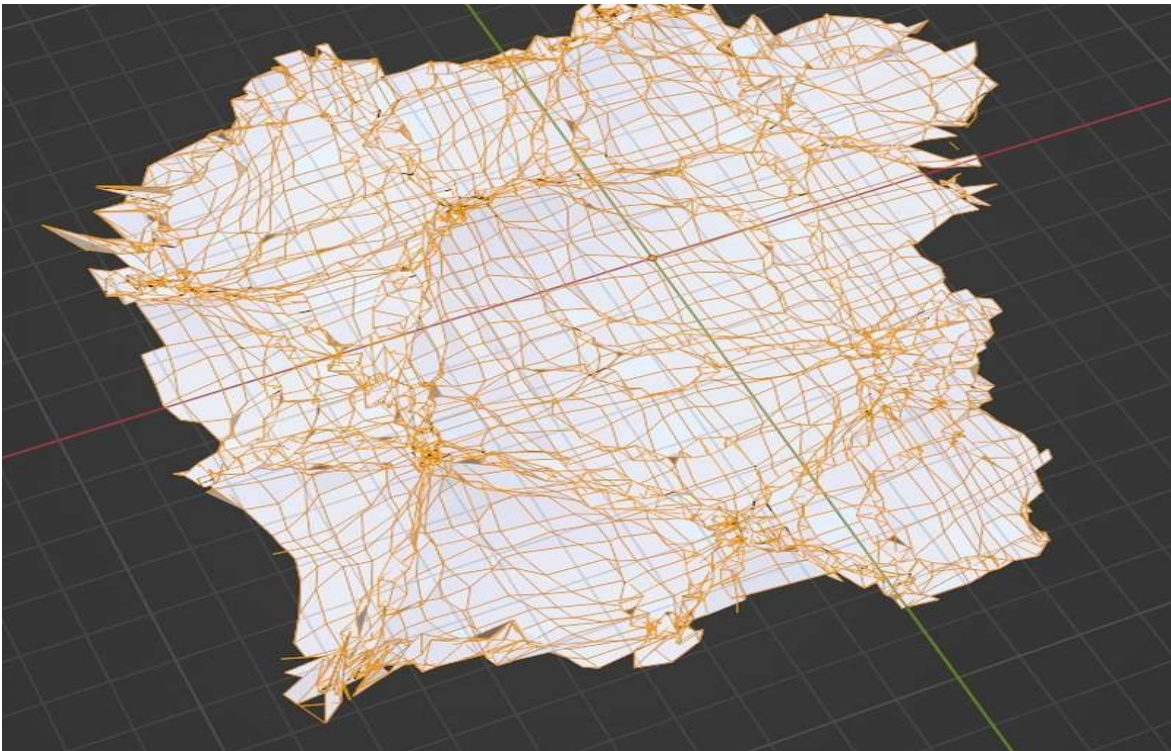


Figure 3.4: Blender-generated mining terrain model illustrating the mesh structure and geometric complexity used for simulation and digital twin representation.

3.6.4 Sim-to-Real Transfer Considerations

The primary sources of sim-to-real gap are: (i) surface reflectance — real mine walls exhibit highly irregular LiDAR reflectance due to moisture and mineral inclusions; (ii) floor

irregularities — actual mine floors contain debris and gradient variations causing wheel slip not present on the simulation's flat ground plane; and (iii) dynamic obstacles — mining equipment, personnel, and falling debris create dynamic occlusions not modelled in the static Gazebo world.

3.8 Adaptive Digital Twin Architecture

3.8.1 Definition

An Adaptive Digital Twin (ADT), in the context of this project, is defined as a live, bidirectionally coupled virtual replica of the physical robot system and its operational environment, which continuously updates its internal state in response to data from physical sensors and, in turn, provides updated world models, predictive diagnostics, and re-planned navigation priors back to the physical system.

3.8.2 Five-Layer Architecture

Layer	Components	Function
Physical Layer	Robot, LiDAR, IMU, encoders, actuators	Real-world sensing and actuation
State Estimation	EKF node, slam_toolbox, /odometry/filtered, /map	Fused pose and map from real sensors
Virtual Layer (Twin)	Gazebo simulation, Blender terrain, Nav2 planner	Predictive simulation and path planning
Adaptation Layer	Map-diff monitor, parameter updater node	Detects sim-real divergence; updates twin parameters
Feedback Layer	ROS 2 bridge (simulation ↔ physical robot)	Synchronises plans and world models bidirectionally

Table 3.4: Five-layer Adaptive Digital Twin architecture and component assignments.

3.8.3 Bidirectional Feedback Loop

The operational cycle of the ADT proceeds as follows:

- The physical robot operates in the mine environment, publishing LiDAR scans, IMU measurements, and odometry to the ROS 2 network.
- The EKF node fuses these measurements into a filtered pose estimate, which slam_toolbox uses to build and update the occupancy grid map (/map).
- Simultaneously, the Gazebo simulation (virtual twin) executes the same navigation plan with an identical virtual robot model.
- The adaptation monitor computes the structural divergence between the real SLAM map and the Gazebo world model.
- The updated Gazebo simulation re-evaluates the current navigation plan and transmits updated waypoints back to the physical robot's Nav2 stack.

3.9 Simulation versus Real-World Performance Analysis

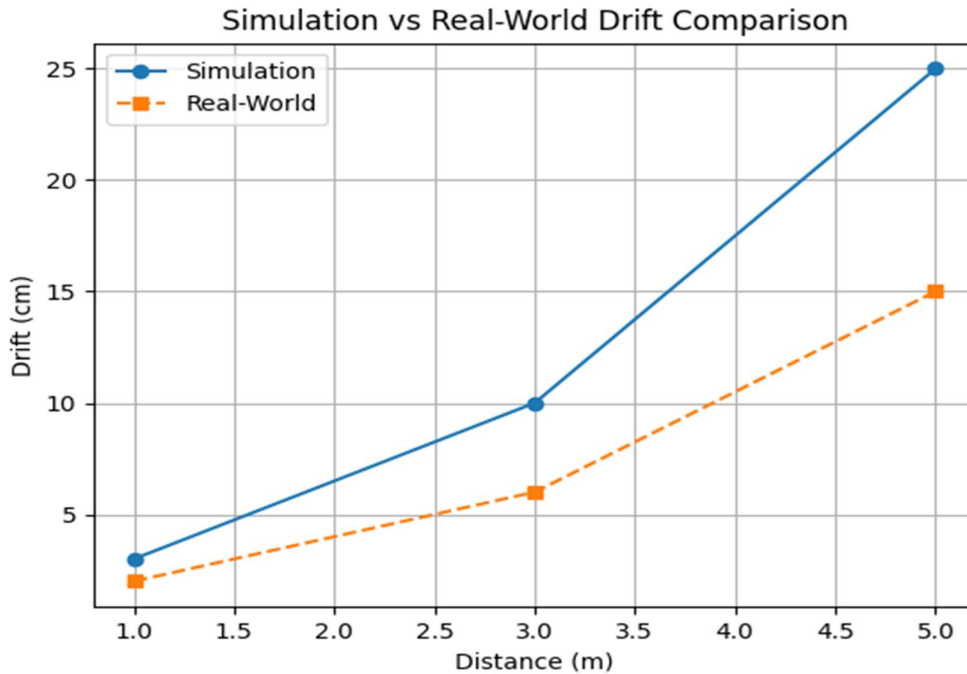


Figure 3.5: Drift Accumulation with Distance — Simulation vs. Real-World Deployment

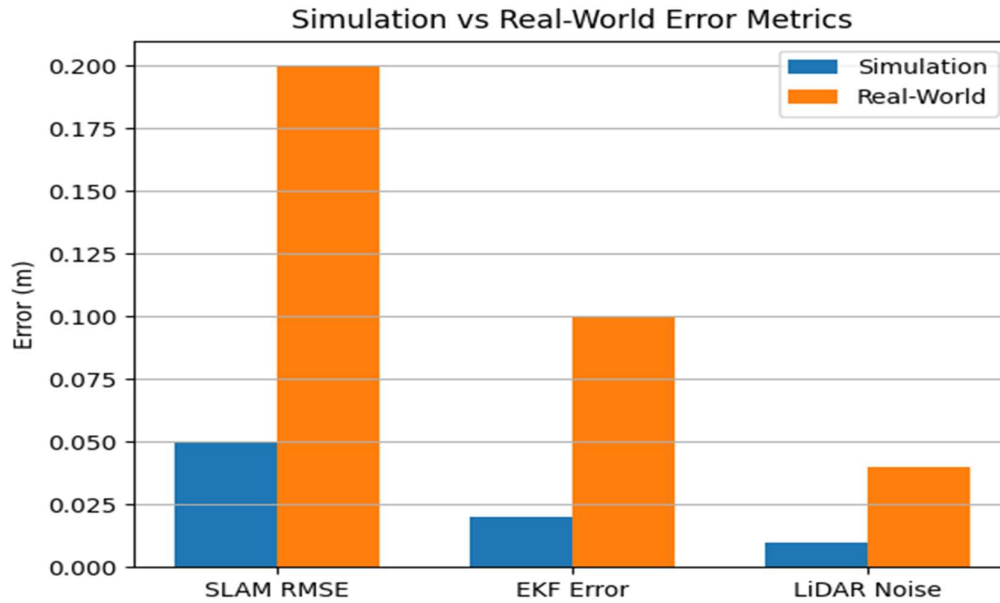


Figure 3.6: Comparative Analysis of Localization and Sensing Errors in Simulation and Real-World Conditions

3.10 Chapter Summary and Statement

This chapter has presented the complete hardware integration and sensing pipeline as the individual engineering contribution to the Digital Adaptive Twin-Driven Planning for Multi-Robot Navigation in GPS-Denied Environments project. The principal contributions are:

- Multi-modal sensor suite design: Integration of 2D LiDAR, IMU, and wheel encoders with a defined ROS 2 topic architecture and per-channel QoS configuration.
- Extended Kalman Filter sensor fusion: Implementation and configuration of the robot_localization EKF node fusing IMU angular velocity and wheel odometry into a continuous filtered pose estimate (/odometry/filtered).
- SLAM mapping pipeline: Deployment and configuration of slam_toolbox in online SLAM mode, producing a 570×880 cell occupancy grid map at 0.05 m resolution.
- Gazebo mining simulation environment: Construction of a Gazebo Classic world replicating underground mine tunnel geometry, lighting, and obstacle layout.
- Blender 3D terrain modelling: Generation of a procedurally displaced and physically solidified 3D terrain mesh representing the surface mining context.
- Adaptive Digital Twin architecture: Design of the five-layer ADT system with a defined bidirectional sim-to-real feedback loop.

4. PROBLEM FORMULATION AND PROPOSED METHODOLOGY

This chapter explains the problem addressed in the present work and the methodology used to solve it. The aim is to develop a coordinated multi-robot navigation system for a mining-inspired environment in which multiple robots must reach their goals safely, avoid collisions with obstacles and one another, and achieve reliable team-level success under realistic sensing and motion constraints.

4.1 Problem Formulation

The problem considered in this work is formulated as a cooperative multi-robot navigation task in a cluttered simulated mining arena. In the implemented setup, three differential-drive robots operate inside a $50\text{ m} \times 50\text{ m}$ PyBullet environment containing walls, pillars, ore piles, and equipment blocks, which together create both geometric and coordination challenges. Each robot is assigned an individual goal and must reach it without colliding with static obstacles or interfering with the progress of other robots in the team.

The task is challenging because each robot does not act from complete global knowledge. Instead, decision-making is based on local observations composed of goal-relative features, LiDAR readings, velocity information, and teammate-related features, so the policy must learn to act from partial yet structured information. In addition, the success of the system is not determined only by whether one robot reaches its goal, but by whether all robots are able to complete the task in a coordinated and efficient manner.

The control setting is continuous, with each robot producing normalized linear and angular commands that are mapped to physical motion in the simulator. The environment also tracks collisions, timeout events, and goal completion, and once a robot reaches its goal it is frozen so that it no longer continues to move. Therefore, the problem is not simply path following, but a cooperative sequential decision-making problem in which safety, perception, coordination, and efficiency must all be learned together.

4.2 Proposed Methodology

To solve this problem in a systematic way, the proposed methodology is organized into five connected modules. These are the Multi-Agent Learning Module, the Simulation and Environment Module, the Reward and Curriculum Module, the Training Infrastructure Module, and the Diagnostics and Evaluation Module. Together, these modules form a complete learning and analysis pipeline for coordinated multi-robot navigation.

4.2.1 Multi-Agent Learning Module

The Multi-Agent Learning Module forms the decision-making core of the system. It is based on a centralized training and decentralized execution MADDPG framework in which each robot acts using its own observations, while the critics are trained using the joint observations and actions of all robots. The implementation includes shared actor architecture, twin critics,

delayed policy updates, target smoothing, soft target updates, gradient clipping, and dead-actor recovery to improve stability during long training runs.

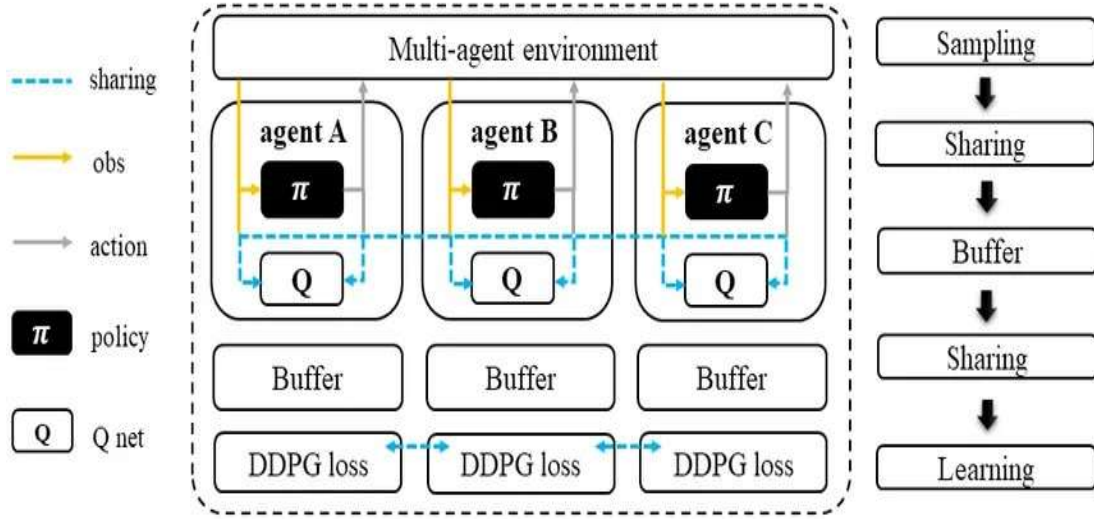


Fig 2: MADDPG Architecture

4.2.2 Simulation and Environment Module

The Simulation and Environment Module provides the virtual world in which policy learning takes place. The PyBullet environment models the mining arena, robot dynamics, obstacle geometry, spawn logic, goal generation, collision handling, and frozen-on-goal semantics. It also provides the perception pipeline through goal-relative states, 16 LiDAR rays, velocity features, and teammate-relative information, allowing each robot to make decisions from meaningful local observations.

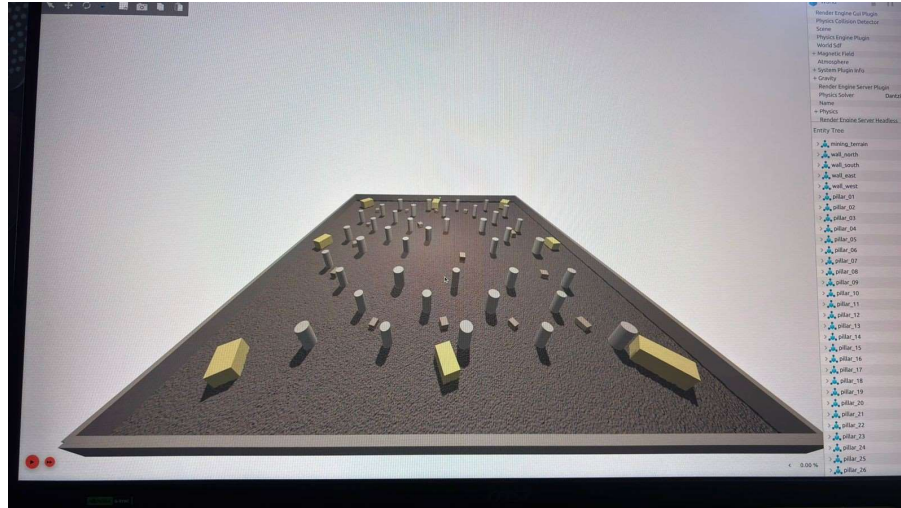


Fig 2: Mining Arena

4.2.3 Reward and Curriculum Module

The Reward and Curriculum Module guides the robots toward safe and cooperative behaviour. The reward structure combines progress-based shaping, near-goal and milestone bonuses, goal rewards, partial and full team bonuses, collision penalties, teammate spacing penalties, and timeout penalties so that complete team success remains the main objective. To make learning more reliable, training

progresses through curriculum phases from Easy1 to VHard, and advancement occurs only when rolling individual and team success both satisfy the 90% mastery condition.

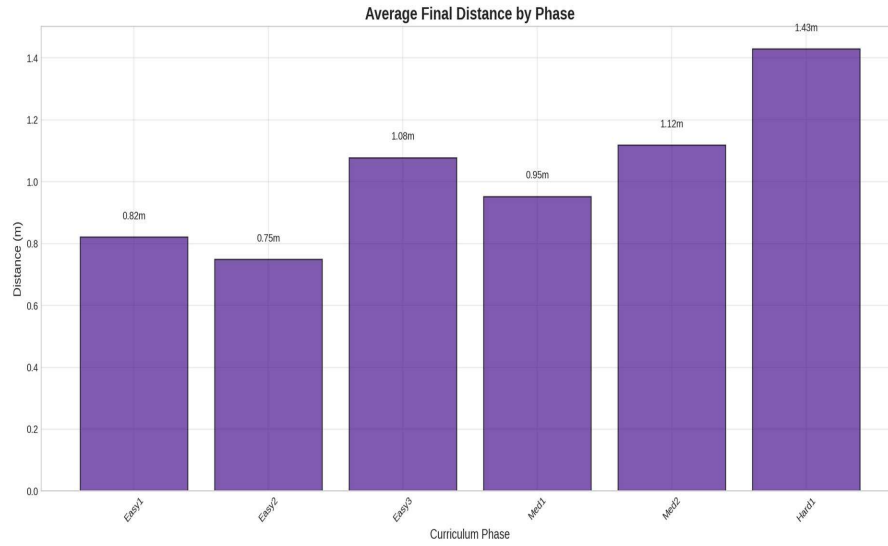


Fig 3: Categories&Mean Distance

4.2.4 Training Infrastructure Modul

The Training Infrastructure Module supports efficient and scalable learning. It includes a replay buffer with list-backed circular storage, a success-oriented memory pool, and phase-indexed sampling so that training remains focused on the current difficulty level without losing experience diversity. This module also supports parallel rollouts, CPU budgeting, checkpointing, and periodic result generation, which are important for running long experiments in a stable and reproducible way

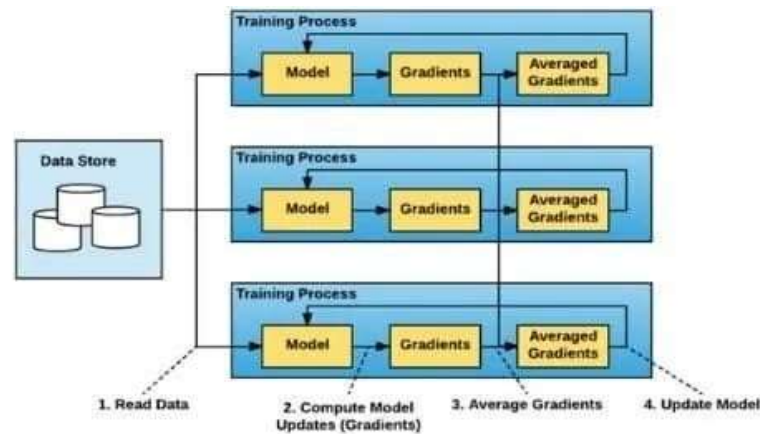


Fig 4:Distributed Learning Architecture

4.2.5 Diagnostics and Evaluation Module

The Diagnostics and Evaluation Module is used to interpret training behaviour and validate performance. During training, the system records reward, individual success, team success, collisions, steps, losses, TD error, action saturation, timeout behaviour, and phase progression, which makes it possible to study both improvement and failure modes in detail. This module therefore provides the evidence needed to judge whether the learned policy has become robust, efficient, and genuinely effective across increasing levels of task difficulty.

5. RESULTS AND DISCUSSION

5.1 Software

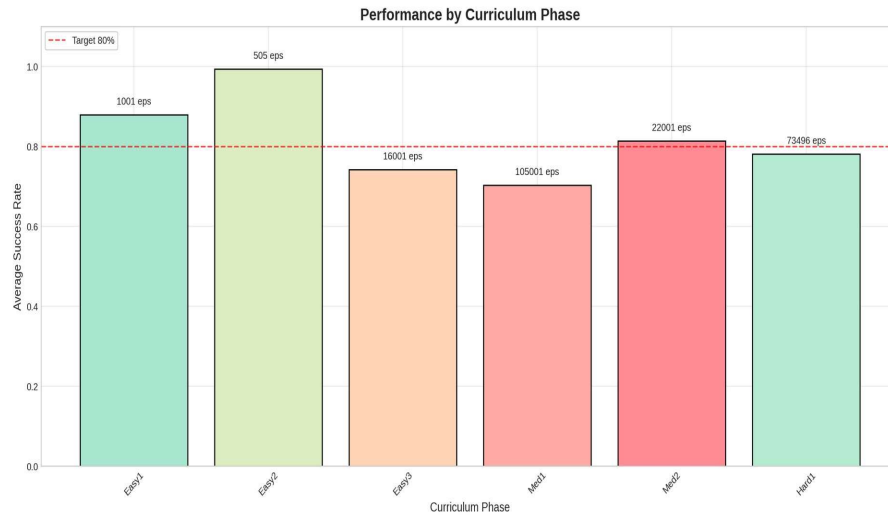


Fig 5: Avg Success Rate

Average success rates per curriculum phase with episode counts annotated. Easy2 achieves 99.6% success (505 eps), triggering advancement, but

Med1 stalls at ~70% (105k eps spent) despite the 80% target line. This quantifies the curriculum's "mastery-only" logic: the system cannot advance to Med2 until both individual and team success exceed 90%, which the plot shows is not yet achieved, explaining the prolonged training time in this phase.

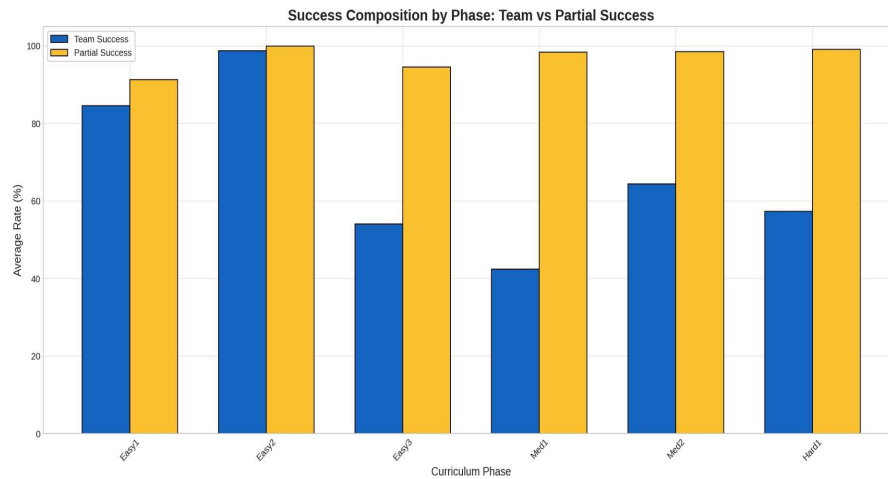


Fig 6: Success Composition

A bar chart exposing the "**Med1 Bottleneck**"—while Partial Success (yellow) stays near 100% across all phases, Team Success (blue) crashes from 98% (Easy2) to 42% (Med1). This gap indicates robots learn individual goal-reaching (0.75m threshold) but fail to synchronize arrivals in the 3–4m Medium-phase range, confirming the curriculum's 90% team success gate is the critical blocking factor for advancement.

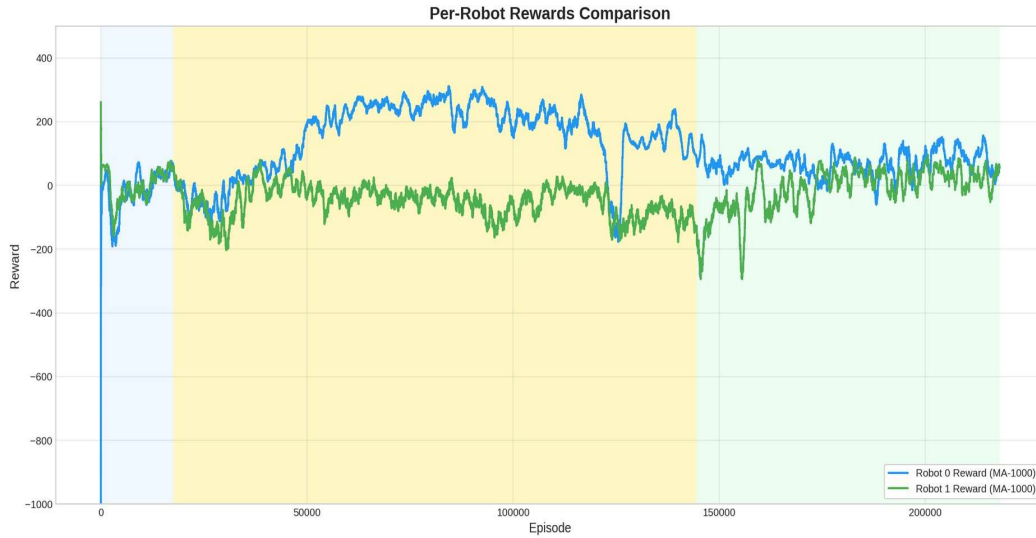


Fig 7: Robot Fairness

Time-series of per-robot moving-average rewards revealing **asymmetric learning progress**: Robot 0 (blue) exploits the early Medium-phase curriculum (yellow background) to achieve higher rewards (~250 MA), while Robot 1 (green) languishes near zero until the Hard phase (green background). This demonstrates the "lead robot" phenomenon in CTDE MADDPG, where one agent's policy collapses into a local optimum until the dead-actor recovery heuristic (copying weights from the best agent) kicks in around episode 140k.

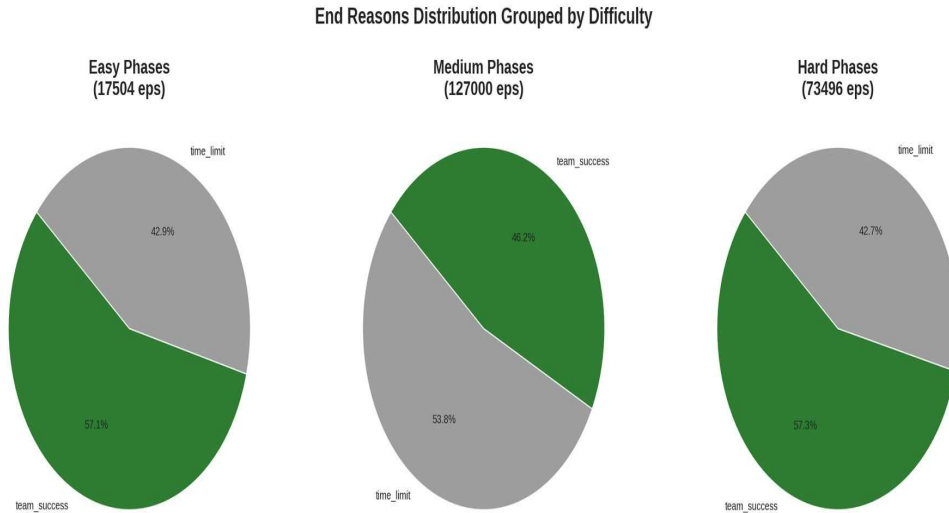


Fig 8: End Reason %

Three pie charts revealing that **Medium phases are the failure mode**: 53.8% of Medium episodes end in time_limit (gray) versus 46.2% team_success (green), whereas Easy and Hard phases achieve >57% team success. This inverted-U pattern suggests the Med1 configuration (0.75m threshold, 3–4m range, 350 steps) creates a "valley of death" where goals are too far for trivial reach yet too close for the robots to fully deploy long-range coordination behaviors learned later.

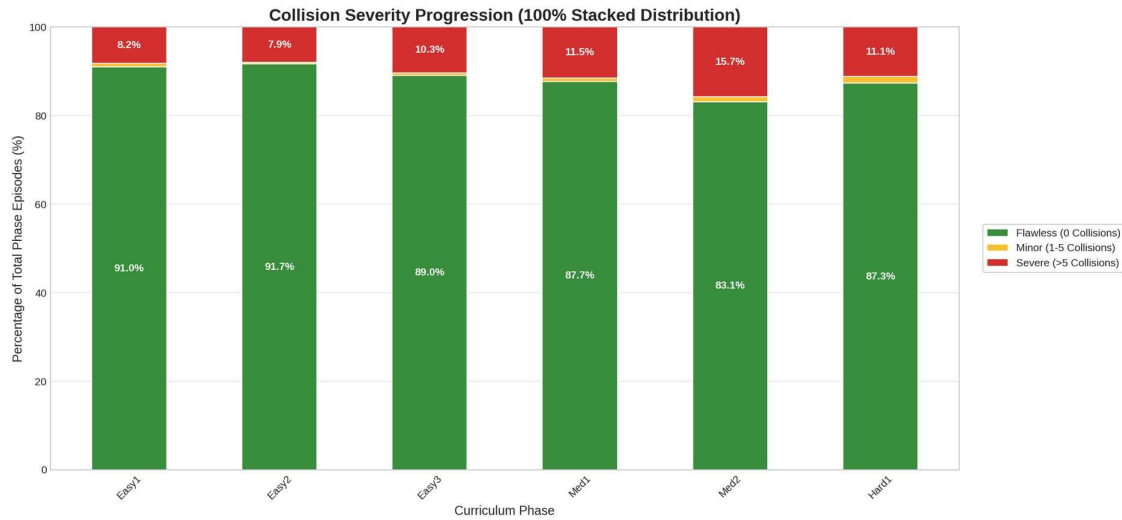


Fig 9: Collision Severity

A 100% stacked distribution showing that despite curriculum difficulty increasing (Easy→Hard), **87–91% of episodes remain collision-free** (green), with severe collisions (>5) peaking at only 15.7% in Med2. This validates the geometric collision penalty (-2.0 reward) and 0.05m safety margins in the PyBullet environment, proving the vectorized LiDAR (16 rays) and anti-collision coordination penalties prevent catastrophic failure even as goal distances stretch to 5.5–6.5m.

6. CONCLUSION AND FUTURE SCOPE

6.1 Conclusion

This work addressed the problem of coordinated multi-robot navigation in a mining-inspired environment by developing a learning-based framework in which multiple robots must reach assigned goals safely, avoid obstacles, and maintain team-level success under realistic sensing and motion constraints. The overall system was not built as an isolated reinforcement learning model, but as a complete software pipeline that combines a PyBullet simulation environment, a CTDE-based MADDPG training framework, a reward-guided curriculum, efficient replay and rollout support, and a diagnostics layer for phase-wise performance analysis.

A major outcome of this work is that coordination was treated as a system design problem rather than only as a policy-learning problem. The environment was designed with realistic geometry, differential-drive robot dynamics, collision checking, LiDAR-based perception, frozen-on-goal semantics, and curriculum-controlled task difficulty, which made the learning problem more meaningful than training in a simplified empty world. On the learning side, the framework used shared actors, twin critics, delayed policy updates, target smoothing, soft target updates, gradient clipping, adaptive exploration, and dead-actor recovery to improve training stability across increasingly difficult phases.

Another important contribution of this work is the close alignment between software design and experimental intent. The reward structure was carefully shaped around progress, milestone completion, individual goal achievement, partial and full team success, collision penalties, and timeout penalties so that the agents were encouraged to complete the actual task rather than exploit dense reward terms. In the same way, the curriculum was organized through phase-wise mastery conditions, requiring rolling individual and team success before progression to harder phases, which made the training process performance-driven rather than arbitrary.

Overall, the work demonstrates that robust multi-robot learning depends on the interaction of environment design, reward structure, training efficiency, and diagnostics, not on the learning algorithm alone. The final framework therefore contributes not only a trained control approach, but also a reconstructable software methodology for studying coordinated navigation in cluttered multi-robot settings.

6.2 Future Scope

Although the current framework establishes a strong foundation, there is clear scope for extending both the learning capability and the realism of the system. At present, the implementation focuses on three robots in a simulated mining arena with continuous control, engineered rewards, and mastery-based curriculum progression, so future work can naturally move toward larger teams, harder coordination settings, and more diverse task objectives. Extending the framework to more robots, denser obstacle layouts, dynamic obstacles, and partially shared task objectives would provide a stronger test of generalization and coordination quality.

There is also strong scope for algorithmic improvement. Future work may compare the present MADDPG-based approach with newer multi-agent methods, incorporate stronger safety constraints during exploration, and study how different replay, curriculum, or reward-design choices affect convergence in difficult phases such as Med1, Med2, Hard1, and beyond. More detailed ablation studies on timeout penalties, catastrophe filtering, actor recovery, and adaptive noise scheduling would also help separate which design choices matter most for stable coordination.

A further direction is to close the gap between simulation and physical deployment. Since the current environment already models robot motion, sensing, and obstacle interaction in a structured way, future work can focus on transferring the trained policies to real robotic platforms through calibration, domain randomization, hardware-aware control limits, and real-world sensing uncertainty. This would make the framework more relevant for practical deployments in mines, warehouses, and other GPS-denied or cluttered environments.

REFERENCES

- [1] R. Lowe, Y. Wu, A. Tamar, J. Harb, P. Abbeel, and I. Mordatch, "Multi-Agent Actor-Critic for Mixed Cooperative-Competitive Environments," NeurIPS, 2017.
- [2] T. P. Lillicrap et al., "Continuous Control with Deep Reinforcement Learning," 2015.
- [3] D. Silver et al., "Deterministic Policy Gradient Algorithms," PMLR, 2014.
- [4] T. Rashid et al., "QMIX: Monotonic Value Function Factorisation for Deep Multi-Agent Reinforcement Learning," ICML/PMLR, 2018.
- [5] C. Mower et al., "ROS-PyBullet Interface: A Framework for Reliable Contact Simulation and Robot Data Collection," PMLR, 2023.
- [6] PyBullet, official project website and simulator documentation.
- [7] T. Moore and D. Stouch, "A Generalized Extended Kalman Filter Implementation for the Robot Operating System," in Proc. Intl. Conf. on Intelligent Autonomous Systems (IAS), 2014.
- [8] S. Macenski, F. Martín, R. White, and J. Clavero, "The Marathon 2: A Navigation System," in Proc. IEEE/RSJ IROS, 2020.
- [9] W. Hess, D. Kohler, H. Rapp, and D. Andor, "Real-Time Loop Closure in 2D LiDAR SLAM," in Proc. IEEE ICRA, 2016.
- [10] R. Siegwart, I. R. Nourbakhsh, and D. Scaramuzza, Introduction to Autonomous Mobile Robots, 2nd ed. MIT Press, 2011.
- [11] S. Macenski, T. Foote, B. Gerkey, C. Lalancette, and W. Woodall, "Robot Operating System 2: Design, Architecture, and Uses in the Wild," Science Robotics, vol. 7, no. 66, 2022.
- [12] N. Koenig and A. Howard, "Design and Use Paradigms for Gazebo, an Open-Source Multi-Robot Simulator," in Proc. IEEE/RSJ IROS, 2004.
- [13] Blender Online Community, Blender — a 3D Modelling and Rendering Package, Blender Foundation. Available: <https://www.blender.org>, 2024.
- [14] M. Grieves, "Digital Twin: Manufacturing Excellence through Virtual Factory Replication," White Paper, 2014.